
CS 335

Web Development

Spring 2025/2026

Milestone 4

Backend API and MongoDB Schema

TEAM Members

NAME: Shaikhah Muneer

ID: 2231172120

NAME: Hana Hato

ID: 2212172700

Project Title:

University Bus Management System

Date	Version	Description	Author/s

I. Backend API Development

1. Overview

The backend API for the **Kuwait University Bus Management System** is built using **Node.js, Express.js, MongoDB, and Mongoose**. Its main role is to connect the React frontend with the MongoDB database.

The backend receives HTTP requests from the frontend, processes the data, communicates with MongoDB, and returns JSON responses. It supports the main system features, including user accounts, buses, routes, trips, bookings, Beeyout subscriptions, reports, notifications, bus status updates, and demand requests.

The React frontend uses this backend to display real data in the Student, Driver, and Admin dashboards. For example, students can view trips, create bookings, submit reports, and manage subscriptions. Drivers can update bus status and send arrival notifications. Admins can manage buses, routes, trips, users, reports, and demand information.

The backend is organized into separate files for better code structure:

- **server.js**: starts the Express server and connects all routes.
- **models/**: contains MongoDB schemas such as User, Bus, Route, Trip, Booking, Subscription, Report, Notification, BusStatusUpdate, and DemandRequest.
- **routes/**: defines API endpoints for each feature.
- **middleware/**: contains authentication middleware used to protect secured routes.
- **controllers/**: contains backend logic for handling requests and responses.

The API follows the RESTful structure using common HTTP methods:

- **GET**: retrieve data from MongoDB.
- **POST**: create new records.
- **PUT/PATCH**: update existing records.
- **DELETE**: remove records

2. Authentication:

The backend uses **JWT-based authentication** to secure private parts of the system. JWT stands for **JSON Web Token**. It is used to verify the identity of the logged-in user when they access protected API routes.

When a user registers or logs in successfully, the backend creates a JWT token. This token contains encoded user information, such as the user ID and role. The token is signed using a secret key stored in the `.env` file.

Example login flow:

1. The user enters email and password in the React frontend.
2. The frontend sends a POST request to `/api/auth/login`.
3. The backend checks if the user exists in MongoDB.
4. The backend compares the entered password with the hashed password stored in the database.
5. If the credentials are correct, the backend creates a JWT token.
6. The frontend stores the token.
7. For protected requests, the frontend sends the token in the request header.

The token is sent like this:

Authorization: Bearer TOKEN_HERE

The authentication middleware checks this header before allowing access to protected routes. If the token is missing, invalid, or expired, the backend rejects the request and returns an unauthorized error.

Protected routes are used for actions such as:

- viewing users
- creating buses
- creating routes
- creating trips
- creating bookings
- creating reports
- creating subscriptions
- sending notifications
- updating bus status

This prevents unauthorized users from manually accessing or changing important system data.

3. API Endpoints

Endpoint 1: Register User

Method: POST

Route: `/api/auth/register`

Description:

Creates a new user account. The backend receives user information, checks if the email already exists, hashes the password, and saves the user in MongoDB.

Request Body:

```
{
  "fullName": "Hana Almutairi",
  "email": "hana@student.edu.kw",
  "password": "Password123",
  "role": "student",
  "universityId": "223100001",
  "gender": "Female"
}
```

Response Success:

```
{
  "message": "User registered successfully",
  "token": "jwt_token_here",
  "user": {
    "id": "userId",
    "fullName": "Hana Almutairi",
    "email": "hana@student.edu.kw",
    "role": "student"
  }
}
```

Error Response:

```
{
  "message": "Email already exists"
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "fullName": "Hana Almutairi",
  "email": "hana@student.edu.kw",
  "password": "Password123",
  "role": "student",
  "universityId": "223100001",
  "gender": "Female"
}'
```

Endpoint 2: Login User

Method: POST

Route: /api/auth/login

Description:

Logs in a user using email and password. If the credentials are correct, the backend generates and returns a JWT token.

Request Body:

```
{
  "email": "hana@student.edu.kw",
  "password": "Password123"
}
```

Response Success:

```
{
  "message": "Login successful",
  "token": "jwt_token_here",
  "user": {
    "id": "userId",
    "email": "hana@student.edu.kw",
    "role": "student"
  }
}
```

Error Response:

```
{
  "message": "Invalid email or password"
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "hana@student.edu.kw",
  "password": "Password123"
}'
```

Endpoint 3: Get All Users

Method: GET

Route: /api/users

Description:

Returns all users stored in the database. This endpoint is mainly used by the admin dashboard.

Request Header:

Authorization: Bearer TOKEN_HERE

Request Body:

None

Response Success:

```
[
  {
    "_id": "userId",
    "fullName": "Hana Almutairi",
    "email": "hana@student.edu.kw",
    "role": "student",
    "universityId": "223100001",
    "gender": "Female"
  }
]
```

Error Response:

```
{
  "message": "Not authorized, no token"
}
```

Sample Call:

```
curl -X GET http://localhost:5000/api/users \
-H "Authorization: Bearer TOKEN_HERE"
```

Endpoint 4: Create Bus

Method: POST**Route:** /api/buses**Description:**

Adds a new bus to the transportation system.

Request Body:

```
{
  "busNumber": "BUS-001",
  "capacity": 40,
  "busType": "College",
  "genderCategory": "Mixed",
  "status": "Available"
}
```

Response Success:

```
{
  "message": "Bus created successfully",
  "bus": {
```

```
{
  "_id": "busId",
  "busNumber": "BUS-001",
  "capacity": 40,
  "busType": "College",
  "genderCategory": "Mixed",
  "status": "Available"
}
```

Error Response:

```
{
  "message": "Failed to create bus"
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/buses \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN_HERE" \
-d '{
  "busNumber": "BUS-001",
  "capacity": 40,
  "busType": "College",
  "genderCategory": "Mixed",
  "status": "Available"
}'
```

Endpoint 5: Get All Buses

Method: GET**Route:** /api/buses**Description:**

Returns all buses available in the system.

Request Body:

None

Response Success:

```
[
  {
    "_id": "busId",
    "busNumber": "BUS-001",
    "capacity": 40,
    "busType": "College",
    "genderCategory": "Mixed",
    "status": "Available"
  }
]
```

Error Response:

```
{
  "message": "Failed to get buses"
}
```

Sample Call:

```
curl -X GET http://localhost:5000/api/buses
```

Endpoint 6: Create Route

Method: POST**Route:** /api/routes**Description:**

Creates a new transportation route between two locations.

Request Body:

```
{
  "routeName": "Shuwaikh to Shadadiya",
  "routeType": "College",
  "departureLocation": "Shuwaikh",
  "destination": "Shadadiya",
  "city": "Kuwait",
  "blockStart": 1,
  "blockEnd": 5
}
```

Response Success:

```
{
  "message": "Route created successfully",
  "route": {
    "_id": "routeId",
    "routeName": "Shuwaikh to Shadadiya",
    "routeType": "College",
    "departureLocation": "Shuwaikh",
    "destination": "Shadadiya"
  }
}
```

Error Response:

```
{
  "message": "Failed to create route"
}
```

Sample Call:


```
curl -X POST http://localhost:5000/api/routes \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer TOKEN_HERE" \  
-d '{  
  "routeName": "Shuwaikh to Shadadiya",  
  "routeType": "College",  
  "departureLocation": "Shuwaikh",  
  "destination": "Shadadiya",  
  "city": "Kuwait",  
  "blockStart": 1,  
  "blockEnd": 5  
}'
```

Endpoint 7: Get All Routes

Method: GET

Route: /api/routes

Description:

Returns all transportation routes.

Request Body:

None

Response Success:

```
[  
  {  
    "_id": "routeId",  
    "routeName": "Shuwaikh to Shadadiya",  
    "routeType": "College",  
    "departureLocation": "Shuwaikh",  
    "destination": "Shadadiya"  
  }  
]
```

Error Response:

```
{  
  "message": "Failed to get routes"  
}
```

Sample Call:

```
curl -X GET http://localhost:5000/api/routes
```

Endpoint 8: Create Trip

Method: POST

Route: /api/trips

Description:

Creates a scheduled bus trip using an existing bus, driver, and route.

Request Body:

```
{
  "busId": "busIdHere",
  "driverId": "driverIdHere",
  "routeId": "routeIdHere",
  "departureTime": "2026-05-10T08:00:00",
  "arrivalTime": "2026-05-10T08:30:00",
  "tripStatus": "Scheduled",
  "availableSeats": 40
}
```

Response Success:

```
{
  "message": "Trip created successfully",
  "trip": {
    "_id": "tripId",
    "busId": "busIdHere",
    "driverId": "driverIdHere",
    "routeId": "routeIdHere",
    "tripStatus": "Scheduled",
    "availableSeats": 40
  }
}
```

Error Response:

```
{
  "message": "Failed to create trip"
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/trips \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN_HERE" \
-d '{
  "busId": "busIdHere",
  "driverId": "driverIdHere",
  "routeId": "routeIdHere",
  "departureTime": "2026-05-10T08:00:00",
  "arrivalTime": "2026-05-10T08:30:00",
  "tripStatus": "Scheduled",
}
```

```
"availableSeats": 40
}
```

Endpoint 9: Get All Trips

Method: GET

Route: /api/trips

Description:

Returns all scheduled trips.

Request Body:

None

Response Success:

```
[
  {
    "_id": "tripId",
    "busId": "busIdHere",
    "driverId": "driverIdHere",
    "routeId": "routeIdHere",
    "departureTime": "2026-05-10T08:00:00",
    "arrivalTime": "2026-05-10T08:30:00",
    "tripStatus": "Scheduled",
    "availableSeats": 40
  }
]
```

Error Response:

```
{
  "message": "Failed to get trips"
}
```

Sample Call:

```
curl -X GET http://localhost:5000/api/trips
```

Endpoint 10: Create Booking

Method: POST

Route: /api/bookings

Description:

Allows a student to book a seat on a selected trip.

Request Body:

```
{
  "studentId": "studentIdHere",
  "tripId": "tripIdHere",
  "bookingStatus": "Confirmed",
  "checkInStatus": "Not Checked In"
}
```

Response Success:

```
{
  "message": "Booking created successfully",
  "booking": {
    "_id": "bookingId",
    "studentId": "studentIdHere",
    "tripId": "tripIdHere",
    "bookingStatus": "Confirmed",
    "checkInStatus": "Not Checked In"
  }
}
```

Error Response:

```
{
  "message": "Failed to create booking"
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/bookings \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN_HERE" \
-d '{
  "studentId": "studentIdHere",
  "tripId": "tripIdHere",
  "bookingStatus": "Confirmed",
  "checkInStatus": "Not Checked In"
}'
```

Endpoint 11: Get User Bookings

Method: GET**Route:** /api/bookings**Description:**

Returns all bookings. For students, it can be used to show their reserved trips.

Request Header:

Authorization: Bearer TOKEN_HERE

Request Body:

None

Response Success:

```
[
  {
    "_id": "bookingId",
    "studentId": "studentIdHere",
    "tripId": "tripIdHere",
    "bookingStatus": "Confirmed",
    "checkInStatus": "Not Checked In"
  }
]
```

Error Response:

```
{
  "message": "Failed to get bookings"
}
```

Sample Call:

```
curl -X GET http://localhost:5000/api/bookings \
-H "Authorization: Bearer TOKEN_HERE"
```

Endpoint 12: Create Subscription

Method: POST**Route:** /api/subscriptions**Description:**

Creates a Beeyout home-service subscription for a student.

Request Body:

```
{
  "studentId": "studentIdHere",
  "homeLocation": "Salmiya",
  "blockNumber": 5,
  "pickupTimeToUniversity": "07:00",
  "pickupTimeToHome": "14:00",
  "serviceType": "Beeyout",
  "paymentAmount": 25,
  "paymentStatus": "Paid",
  "subscriptionStatus": "Active"
}
```

Response Success:

```
{
  "message": "Subscription created successfully",
  "subscription": {
    "_id": "subscriptionId",
    "studentId": "studentIdHere",
    "homeLocation": "Salmiya",
    "blockNumber": 5,
    "serviceType": "Beeyout",
    "paymentStatus": "Paid",
    "subscriptionStatus": "Active"
  }
}
```

Error Response:

```
{
  "message": "Failed to create subscription"
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/subscriptions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN_HERE" \
-d '{
  "studentId": "studentIdHere",
  "homeLocation": "Salmiya",
  "blockNumber": 5,
  "pickupTimeToUniversity": "07:00",
  "pickupTimeToHome": "14:00",
  "serviceType": "Beeyout",
  "paymentAmount": 25,
  "paymentStatus": "Paid",
  "subscriptionStatus": "Active"
}'
```

Endpoint 13: Get Subscriptions

Method: GET**Route:** /api/subscriptions**Description:**

Returns subscription records from the database. Admin can use it to view student Beeyout subscriptions.

Request Body:

None

Response Success:

```
[
  {
    "_id": "subscriptionId",
    "studentId": "studentIdHere",
    "homeLocation": "Salmiya",
    "blockNumber": 5,
    "serviceType": "Beeyout",
    "paymentStatus": "Paid",
    "subscriptionStatus": "Active"
  }
]
```

Error Response:

```
{
  "message": "Failed to get subscriptions"
}
```

Sample Call:

```
curl -X GET http://localhost:5000/api/subscriptions \
-H "Authorization: Bearer TOKEN_HERE"
```

Endpoint 14: Create Report

Method: POST**Route:** /api/reports**Description:**

Allows a student to submit a transportation complaint or issue report.

Request Body:

```
{
  "studentId": "studentIdHere",
  "routeId": "routeIdHere",
  "description": "The bus arrived late",
  "reportStatus": "Pending"
}
```

Response Success:

```
{
  "message": "Report created successfully",
  "report": {
    "_id": "reportId",
    "studentId": "studentIdHere",
    "routeId": "routeIdHere",
    "description": "The bus arrived late",
    "reportStatus": "Pending"
  }
}
```

```
}  
}
```

Error Response:

```
{  
  "message": "Failed to create report"  
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/reports \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer TOKEN_HERE" \  
-d '{  
  "studentId": "studentIdHere",  
  "routeId": "routeIdHere",  
  "description": "The bus arrived late",  
  "reportStatus": "Pending"  
}'
```

Endpoint 15: Get Reports

Method: GET**Route:** /api/reports**Description:**

Returns all submitted student reports.

Request Body:

None

Response Success:

```
[  
  {  
    "_id": "reportId",  
    "studentId": "studentIdHere",  
    "routeId": "routeIdHere",  
    "description": "The bus arrived late",  
    "reportStatus": "Pending"  
  }  
]
```

Error Response:

```
{  
  "message": "Failed to get reports"  
}
```


Sample Call:

```
curl -X GET http://localhost:5000/api/reports \
-H "Authorization: Bearer TOKEN_HERE"
```

Endpoint 16: Create Notification

Method: POST**Route:** /api/notifications**Description:**

Creates a notification between users. For example, a driver can notify a student that the bus has arrived.

Request Body:

```
{
  "senderId": "driverIdHere",
  "receiverId": "studentIdHere",
  "message": "Driver has arrived at your pickup location",
  "notificationType": "Arrival",
  "readStatus": false
}
```

Response Success:

```
{
  "message": "Notification created successfully",
  "notification": {
    "_id": "notificationId",
    "senderId": "driverIdHere",
    "receiverId": "studentIdHere",
    "message": "Driver has arrived at your pickup location",
    "notificationType": "Arrival",
    "readStatus": false
  }
}
```

Error Response:

```
{
  "message": "Failed to create notification"
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/notifications \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN_HERE" \
-d '{
```

```
"senderId": "driverIdHere",  
"receiverId": "studentIdHere",  
"message": "Driver has arrived at your pickup location",  
"notificationType": "Arrival",  
"readStatus": false  
}'
```

Endpoint 17: Get Notifications

Method: GET

Route: /api/notifications

Description:

Returns notifications for users.

Request Header:

Authorization: Bearer TOKEN_HERE

Request Body:

None

Response Success:

```
[  
  {  
    "_id": "notificationId",  
    "senderId": "driverIdHere",  
    "receiverId": "studentIdHere",  
    "message": "Driver has arrived at your pickup location",  
    "notificationType": "Arrival",  
    "readStatus": false  
  }  
]
```

Error Response:

```
{  
  "message": "Failed to get notifications"  
}
```

Sample Call:

```
curl -X GET http://localhost:5000/api/notifications \  
-H "Authorization: Bearer TOKEN_HERE"
```

Endpoint 18: Update Bus Status

Method: POST

Route: /api/bus-status-updates

Description:

Stores bus status and location updates sent by the driver. This helps the frontend show bus movement/status information.

Request Body:

```
{
  "busId": "busIdHere",
  "driverId": "driverIdHere",
  "currentStatus": "In Transit",
  "currentLocation": "Block 5, Salmiya"
}
```

Response Success:

```
{
  "message": "Bus status updated successfully",
  "statusUpdate": {
    "_id": "statusId",
    "busId": "busIdHere",
    "driverId": "driverIdHere",
    "currentStatus": "In Transit",
    "currentLocation": "Block 5, Salmiya"
  }
}
```

Error Response:

```
{
  "message": "Failed to update bus status"
}
```

Sample Call:

```
curl -X POST http://localhost:5000/api/bus-status-updates \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN_HERE" \
-d '{
  "busId": "busIdHere",
  "driverId": "driverIdHere",
  "currentStatus": "In Transit",
  "currentLocation": "Block 5, Salmiya"
}'
```

Endpoint 19: Get Bus Status Updates

Method: GET

Route: /api/bus-status-updates

Description:

Returns the latest bus status and location updates.

Request Body:

None

Response Success:

```
[
  {
    "_id": "statusId",
    "busId": "busIdHere",
    "driverId": "driverIdHere",
    "currentStatus": "In Transit",
    "currentLocation": "Block 5, Salmiya",
    "updatedAt": "2026-05-10T08:10:00.000Z"
  }
]
```

Error Response:

```
{
  "message": "Failed to get bus status updates"
}
```

Sample Call:

```
curl -X GET http://localhost:5000/api/bus-status-updates \
-H "Authorization: Bearer TOKEN_HERE"
```

II. Revised Database Schema

Collection 1: User

- **fullName:** String, required, stores the user's full name.
- **email:** String, required, unique, lowercase, stores the user's email.
- **phone:** String, stores the user's phone number.
- **password:** String, required, minimum 6 characters, stored as a hashed password.
- **role:** String, can be student, driver, or admin. Default is student.
- **universityId:** String, stores the student/employee university ID.
- **gender:** String, can be female or male. Default is female.
- **city:** String, stores user city.
- **block:** String, stores user block number.
- **resetPasswordOtp:** String, stores password reset OTP.
- **resetPasswordOtpExpires:** Date, stores OTP expiration time.
- **isActive:** Boolean, default is true.

Indexes:

Email has a unique index.

Relationships:

User is connected to Trip as driver, Booking as student, Subscription as student, Report as student, Notification as sender/receiver, DemandRequest as student, and BusStatusUpdate as driver.

Collection 2: Bus

- **busNumber:** String, required, unique, stores the bus number.
- **capacity:** Number, required, minimum 1, stores bus seat capacity.
- **busType:** String, required, can be beeyout, campus-off, or college.
- **genderCategory:** String, can be female-only or mixed. Default is mixed.
- **status:** String, can be available, assigned, maintenance, or inactive.
- **isActive:** Boolean, default is true.

Indexes:

Bus number has a unique index.

Relationships:

Bus is referenced by Trip and BusStatusUpdate.

Collection 3: Route

- **routeName:** String, required, stores the route name.
- **routeType:** String, required, can be beeyout, campus-off, or college.
- **departureLocation:** String, required, stores starting location.
- **destination:** String, required, stores destination.
- **city:** String, stores city name.
- **blockStart:** Number, stores starting block for route coverage.
- **blockEnd:** Number, stores ending block for route coverage.
- **stops:** Array, stores route stop information.
 - **name:** String, stop name.
 - **order:** Number, stop order.
 - **latitude:** Number, stop latitude.
 - **longitude:** Number, stop longitude.
- **isActive:** Boolean, default is true.

Indexes:

No custom indexes are defined.

Relationships:

Route is referenced by Trip and Report.

Collection 4: Trip

- **bus:** ObjectId, required, references Bus.
- **driver:** ObjectId, required, references User.
- **route:** ObjectId, references Route, default is null.
- **governorate:** String, stores governorate name.
- **area:** String, stores area name.
- **departureTime:** Date, required, stores trip departure time.
- **arrivalTime:** Date, required, stores trip arrival time.
- **tripStatus:** String, can be scheduled, in-transit, arrived, pickup-complete, completed, or cancelled.
- **availableSeats:** Number, required, minimum 0.
- **currentOccupancy:** Number, default 0, minimum 0.
- **arrivedAt:** Date, stores driver arrival time.
- **pickupCompletedAt:** Date, stores pickup completion time.
- **completedAt:** Date, stores trip completion time.
- **notes:** String, stores extra trip notes.

Indexes:

No custom indexes are defined.

Relationships:

Trip references Bus, User as driver, and Route. Trip is also referenced by Booking, Report, and DemandRequest.

Collection 5: Booking

- **student:** ObjectId, required, references User.
- **trip:** ObjectId, required, references Trip.
- **bookingStatus:** String, can be booked, cancelled, or completed. Default is booked.
- **checkInStatus:** String, can be not-checked-in, checked-in, or checked-out.
- **checkInTime:** Date, stores check-in time.
- **checkOutTime:** Date, stores check-out time.
- **qrCodeValue:** String, stores QR code value for booking/check-in.

Indexes:

Unique compound index on student and trip, so the same student cannot book the same trip twice.

Relationships:

Booking references User as student and Trip.

Collection 6: Subscription

- **student:** ObjectId, required, references User.
- **homeLocation:** String, required, stores student home area/location.
- **blockNumber:** String, required, stores student block number.
- **pickupTimeToUniversity:** String, stores pickup time going to university.
- **pickupTimeToHome:** String, stores pickup time going back home.
- **serviceType:** String, only beeyout. Default is beeyout.
- **paymentAmount:** Number, default is 0.
- **paymentStatus:** String, can be unpaid, paid, or refunded.
- **subscriptionStatus:** String, can be active, paused, cancelled, or expired.

Indexes:

No custom indexes are defined.

Relationships:

Subscription references User as student.

Collection 7: Notification

- **sender:** ObjectId, references User.
- **receiver:** ObjectId, required, references User.
- **message:** String, required, stores notification message.
- **notificationType:** String, can be arrival, booking, system, report, or subscription.
- **readStatus:** Boolean, default is false.

Indexes:

No custom indexes are defined.

Relationships:

Notification references User as sender and User as receiver.

Collection 8: Report

- **student:** ObjectId, required, references User.
- **route:** ObjectId, references Route.
- **trip:** ObjectId, references Trip.
- **description:** String, required, stores issue/report description.
- **reportStatus:** String, can be open, in-review, resolved, or rejected.

Indexes:

No custom indexes are defined.

Relationships:

Report references User as student, Route, and Trip.

Collection 9: BusStatusUpdate

- **bus:** ObjectId, required, references Bus.
- **driver:** ObjectId, required, references User.
- **currentStatus:** String, required, can be available, in-transit, arrived, maintenance, or inactive.
- **currentLocation:** String, stores current bus location.

Indexes:

No custom indexes are defined.

Relationships:

BusStatusUpdate references Bus and User as driver.

Collection 10: DemandRequest

- **student:** ObjectId, required, references User.
- **trip:** ObjectId, required, references Trip.
- **requestStatus:** String, can be pending, approved, or rejected.
- **requestTime:** Date, default is current date/time.

Indexes:

No custom indexes are defined.

Relationships:

DemandRequest references User as student and Trip.

III. Test cases

Test Case 1: Register User Successfully

- **Objective:** Verify that a new user can create an account using valid data.
- **Method:** POST /api/auth/register
- **Input:**

```
{
  "fullName": "Hana Almutairi",
  "email": "hana@student.edu.kw",
  "password": "Password123",
  "role": "student",
  "universityId": "223100001",
  "gender": "female"
}
```

- **Expected Output:** User account is created and saved in MongoDB. The response returns a success message and JWT token.
 - **Result:** Passed
-

Test Case 2: Login User Successfully

- **Objective:** Verify that a registered user can log in with correct credentials.
- **Method:** POST /api/auth/login
- **Input:**

```
{
  "email": "hana@student.edu.kw",
  "password": "Password123"
}
```

- **Expected Output:** Backend returns login success message, user information, and JWT token.
 - **Result:** Passed
-

Test Case 3: Reject Login With Wrong Password

- **Objective:** Verify that the backend rejects incorrect login credentials.
- **Method:** POST /api/auth/login
- **Input:**

```
{  
  "email": "hana@student.edu.kw",  
  "password": "WrongPassword123"  
}
```

- **Expected Output:** Backend returns an error message such as "Invalid email or password".
 - **Result:** Passed
-

Test Case 4: Access Protected Route Without Token

- **Objective:** Verify that protected routes cannot be accessed without JWT token.
 - **Method:** GET /api/users
 - **Input:** No Authorization header.
 - **Expected Output:** Backend returns 401 Unauthorized with message "Not authorized, no token".
 - **Result:** Passed
-

Test Case 5: Create Bus Successfully

- **Objective:** Verify that admin can add a new bus to the system.
- **Method:** POST /api/buses
- **Input:**

```
{  
  "busNumber": "BUS-001",  
  "capacity": 40,  
  "busType": "college",  
  "genderCategory": "mixed",  
  "status": "available"  
}
```

- **Expected Output:** Bus is created and stored in MongoDB.
 - **Result:** Passed
-

Test Case 6: Get All Buses

- **Objective:** Verify that the system can retrieve all bus records.
 - **Method:** GET /api/buses
 - **Input:** None.
 - **Expected Output:** Backend returns an array of buses.
 - **Result:** Passed
-

Test Case 7: Create Route Successfully

- **Objective:** Verify that admin can create a transportation route.
- **Method:** POST /api/routes
- **Input:**

```
{
  "routeName": "Shuwaikh to Shadadiya",
  "routeType": "college",
  "departureLocation": "Shuwaikh",
  "destination": "Shadadiya",
  "city": "Kuwait",
  "blockStart": 1,
  "blockEnd": 5
}
```

- **Expected Output:** Route is created and saved in MongoDB.
 - **Result:** Passed
-

Test Case 8: Create Trip Successfully

- **Objective:** Verify that admin can create a scheduled trip using a bus, driver, and route.
- **Method:** POST /api/trips
- **Input:**

```
{
  "bus": "busIdHere",
  "driver": "driverIdHere",
  "route": "routeIdHere",
  "departureTime": "2026-05-10T08:00:00",
  "arrivalTime": "2026-05-10T08:30:00",
}
```

```
"tripStatus": "scheduled",  
"availableSeats": 40  
}
```

- **Expected Output:** Trip is created and connected to bus, driver, and route.
 - **Result:** Passed
-

Test Case 9: Get All Trips

- **Objective:** Verify that students and admins can view available scheduled trips.
 - **Method:** GET /api/trips
 - **Input:** None.
 - **Expected Output:** Backend returns an array of trips with bus, driver, route, time, status, and available seats.
 - **Result:** Passed
-

Test Case 10: Create Booking Successfully

- **Objective:** Verify that a student can book a seat on a trip.
- **Method:** POST /api/bookings
- **Input:**

```
{  
  "student": "studentIdHere",  
  "trip": "tripIdHere",  
  "bookingStatus": "booked",  
  "checkInStatus": "not-checked-in"  
}
```

- **Expected Output:** Booking is created and saved in MongoDB.
 - **Result:** Passed
-

Test Case 11: Prevent Duplicate Booking

- **Objective:** Verify that the same student cannot book the same trip twice.
 - **Method:** POST /api/bookings
 - **Input:** Same student and same trip as an existing booking.
 - **Expected Output:** Backend rejects duplicate booking because of the unique student-trip index.
 - **Result:** Passed
-

Test Case 12: Create Beeyout Subscription

- **Objective:** Verify that a student can create a Beeyout home-service subscription.
- **Method:** POST /api/subscriptions
- **Input:**

```
{  
  "student": "studentIdHere",  
  "homeLocation": "Salmiya",  
  "blockNumber": "5",  
  "pickupTimeToUniversity": "07:00",  
  "pickupTimeToHome": "14:00",  
  "serviceType": "beeyout",  
  "paymentAmount": 25,  
  "paymentStatus": "paid",  
  "subscriptionStatus": "active"  
}
```

- **Expected Output:** Subscription is created and connected to the student.
 - **Result:** Passed
-

Test Case 13: Create Report Successfully

- **Objective:** Verify that a student can submit a transportation issue report.
- **Method:** POST /api/reports
- **Input:**

```
{  
  "student": "studentIdHere",  
  "route": "routeIdHere",  
  "trip": "tripIdHere",  
  "description": "The bus arrived late",  
  "reportStatus": "open"  
}
```

- **Expected Output:** Report is saved with status open.
 - **Result:** Passed
-

Test Case 14: Create Notification Successfully

- **Objective:** Verify that the system can create a notification for a user.
- **Method:** POST /api/notifications
- **Input:**

```
{  
  "sender": "driverIdHere",
```

```
"receiver": "studentIdHere",  
"message": "Driver has arrived at your pickup location",  
"notificationType": "arrival",  
"readStatus": false  
}
```

- **Expected Output:** Notification is saved and linked to sender and receiver.
 - **Result:** Passed
-

Test Case 15: Update Bus Status Successfully

- **Objective:** Verify that a driver can update the current bus status and location.
- **Method:** POST /api/bus-status-updates
- **Input:**

```
{  
  "bus": "busIdHere",  
  "driver": "driverIdHere",  
  "currentStatus": "in-transit",  
  "currentLocation": "Block 5, Salmiya"  
}
```

- **Expected Output:** Bus status update is stored in MongoDB.
 - **Result:** Passed
-

Test Case 16: Create Demand Request

- **Objective:** Verify that a student can request demand for a trip.
- **Method:** POST /api/demand-requests
- **Input:**

```
{  
  "student": "studentIdHere",  
  "trip": "tripIdHere",  
  "requestStatus": "pending"  
}
```

- **Expected Output:** Demand request is created with status pending.
- **Result:** Passed

IV. Contributions

Team member	Contribution	
	Who did what	Percentage (%)
Hana Hato	Worked on the database schema design, completed most of the frontend UI implementation, and tested many frontend UI pages and components.	50%
Shaikhah Muneer	Worked on most of the backend implementation, completed backend testing/final API testing, and helped with parts of the database schema	50%